

Dynamic Programming

Nama: Alya Putri Avianti (25/566136/PPA/07134)

1 *Coin-Row Problem*

1.1 Definisi Masalah

Diberikan sebuah barisan yang terdiri dari n koin dengan nilai bilangan bulat positif $c_1, c_2, \dots, c_n$. Nilai-nilai koin tidak harus berbeda (bisa ada yang sama).

Tujuan: Memilih sekumpulan koin dari barisan tersebut untuk mendapatkan **jumlah nilai maksimum**.

Kendala (Constraint): Tidak diperbolehkan mengambil dua koin yang posisinya **bersebelahan** (*adjacent*) dalam barisan awal.

1.2 Penurunan Relasi Rekurensi

Misalkan $F(n)$ adalah jumlah uang maksimum yang dapat diperoleh dari n koin pertama. Untuk menentukan nilai optimal $F(n)$, terdapat dua kemungkinan keputusan yang saling lepas (*mutually exclusive*) terhadap koin terakhir (c_n):

1. Koin ke- n diambil:

Jika koin ke- n diambil, maka koin ke- $(n-1)$ tidak boleh diambil karena kendala posisi bersebelahan. Oleh karena itu, sisa nilai maksimum harus diperoleh dari sub-masalah hingga koin ke- $(n-2)$.

$$\text{Nilai} = c_n + F(n-2)$$

2. Koin ke- n tidak diambil:

Jika koin ke- n diabaikan, maka solusi optimal sama dengan solusi optimal untuk $n-1$ koin pertama.

$$\text{Nilai} = F(n-1)$$

Karena tujuannya adalah memaksimalkan jumlah, maka dipilih nilai terbesar dari kedua opsi di atas.

Rumus Rekurensi:

$$F(n) = \max\{c_n + F(n-2), F(n-1)\} \quad \text{untuk } n > 1 \quad (1)$$

Kondisi Dasar (Base Cases):

- $F(0) = 0$ (Tidak ada koin, nilai maksimum adalah 0).
- $F(1) = c_1$ (Hanya satu koin, koin tersebut diambil).

1.3 Ilustrasi Algoritma

Diberikan contoh himpunan koin: $C = \{5, 1, 2, 10, 6, 2\}$.

Tabel berikut menunjukkan proses pengisian nilai $F(n)$ secara *bottom-up*:

Indeks (i)	Nilai Koin (c_i)	Perhitungan: $\max(c_i + F[i - 2], F[i - 1])$	Nilai $F[i]$
0	-	-	0
1	5	$c_1 = 5$	5
2	1	$\max(1 + 0, 5) = 5$	5
3	2	$\max(2 + 5, 5) = 7$	7
4	10	$\max(10 + 5, 7) = 15$	15
5	6	$\max(6 + 7, 15) = 15$	15
6	2	$\max(2 + 15, 15) = 17$	17

Solusi Maksimum: 17.

1.4 Rekonstruksi Solusi (Backtracing)

Untuk mengetahui koin mana saja yang menyusun nilai 17, dilakukan penelusuran balik dari $F[6]$:

- $F[6] = 17$. Nilai ini berasal dari $c_6 + F[4]$ ($2 + 15$), bukan dari $F[5]$. → **Pilih c_6 (2).**
- Mundur ke $F[4] = 15$. Nilai ini berasal dari $c_4 + F[2]$ ($10 + 5$), bukan dari $F[3]$. → **Pilih c_4 (10).**
- Mundur ke $F[2] = 5$. Nilai ini berasal dari $F[1]$, bukan $c_2 + F[0]$. → c_2 tidak dipilih.
- Mundur ke $F[1] = 5$. → **Pilih c_1 (5).**

Koin terpilih: $\{5, 10, 2\}$.

2 Change-Making Problem

2.1 Definisi Masalah

Diberikan jumlah uang n dan himpunan pecahan koin (denominasi) $d_1, d_2, \dots, d_m$ dengan $d_1 < d_2 < \dots < d_m$.

Asumsi: $d_1 = 1$ (agar solusi selalu memungkinkan untuk sebarang bilangan bulat n).

Tujuan: Menentukan jumlah keping koin **paling sedikit (minimum)** yang diperlukan untuk mencapai jumlah tepat n .

2.2 Penurunan Relasi Rekurensi

Misalkan $F(n)$ adalah jumlah minimum koin yang diperlukan untuk mencapai nilai n . Untuk mencapai nilai n , langkah terakhir pasti melibatkan penambahan satu keping koin dari jenis pecahan yang tersedia (d_j). Jika koin d_j dipilih, maka sisa nilai yang harus dipenuhi adalah $n - d_j$.

Masalah ini dapat direduksi menjadi mencari nilai minimum dari semua kemungkinan koin terakhir yang valid.

Rumus Rekurensi:

$$F(n) = \min_{j:n \geq d_j} \{F(n - d_j)\} + 1 \quad \text{untuk } n > 0 \quad (2)$$

Kondisi Dasar (Base Case): $F(0) = 0$ (Jumlah uang 0 membutuhkan 0 keping koin).

2.3 Ilustrasi Algoritma

Diberikan target $n = 6$ dengan pecahan koin $D = \{1, 3, 4\}$.

Proses pengisian tabel $F(n)$:

$n = 1$ Hanya bisa pakai koin 1.

$$F[1] = F[0] + 1 = 1.$$

$n = 2$ Hanya bisa pakai koin 1.

$$F[2] = F[1] + 1 = 2.$$

$n = 3$ Bisa pakai koin 1 atau 3.

$$\text{Koin 1: } F[2] + 1 = 3$$

$$\text{Koin 3: } F[0] + 1 = 1 \text{ (Minimum)}$$

$$F[3] = 1.$$

$n = 4$ Bisa pakai koin 1, 3, atau 4.

$$\text{Koin 1: } F[3] + 1 = 2$$

$$\text{Koin 3: } F[1] + 1 = 2$$

$$\text{Koin 4: } F[0] + 1 = 1 \text{ (Minimum)}$$

$$F[4] = 1.$$

$n = 5$ Bisa pakai koin 1, 3, atau 4.

$$\text{Koin 1: } F[4] + 1 = 2$$

$$\text{Koin 3: } F[2] + 1 = 3$$

$$\text{Koin 4: } F[1] + 1 = 2$$

$$F[5] = 2 \text{ (Pilih koin 1 atau 4 sama baiknya).}$$

$n = 6$ Bisa pakai koin 1, 3, atau 4.

$$\text{Koin 1: } F[5] + 1 = 3$$

$$\text{Koin 3: } F[3] + 1 = 1 + 1 = 2 \text{ (Minimum)}$$

$$\text{Koin 4: } F[2] + 1 = 3$$

$$F[6] = 2.$$

Hasil Akhir: Minimum 2 keping koin.

2.4 Rekonstruksi Solusi (Backtracing)

Untuk $n = 6$, nilai minimum diperoleh dari opsi "Koin 3" (merujuk ke $F[3]$).

- Koin pertama: 3. Sisa $6 - 3 = 3$.
- Untuk $n = 3$, nilai minimum diperoleh dari opsi "Koin 3" (merujuk ke $F[0]$).
- Koin kedua: 3. Sisa $3 - 3 = 0$.

Komposisi koin optimal: dua keping pecahan 3 ($3 + 3 = 6$).

3 Analisis Kompleksitas

Kedua algoritma di atas menggunakan pendekatan tabel (*tabulation*) satu dimensi.

3.1 Coin-Row Problem

- **Kompleksitas Waktu:** $\Theta(n)$. Diperlukan satu kali iterasi untuk mengisi tabel sepanjang n .
- **Kompleksitas Ruang:** $\Theta(n)$ untuk menyimpan array F . (Dapat dioptimasi menjadi $\Theta(1)$ karena hanya nilai $F[i - 1]$ dan $F[i - 2]$ yang diperlukan pada setiap langkah).

3.2 Change-Making Problem

- **Kompleksitas Waktu:** $O(nm)$. Untuk setiap nilai n (dari 1 sampai target), algoritma harus memeriksa m jenis pecahan koin.
- **Kompleksitas Ruang:** $\Theta(n)$ untuk menyimpan array F agar dapat melakukan *look-up* nilai sebelumnya.

Kesimpulan

Kedua problem ini memberikan contoh implementasi *Dynamic Programming* dalam menyelesaikan masalah optimasi dengan menghindari perhitungan ulang melalui penyimpanan hasil sub-masalah dalam memori (tabel).